

1 Introduction to Lean

- Functional programming language and theorem prover
- Designed for formal verification tasks
- Based on dependent type theory
- Open-source, first developed by Microsoft
- First launched in 2013; Lean 4 released in 2021
- In 2026, three Erdős conjectures were resolved in Lean

2 Mathlib

- Library (or package) for Lean
- Repository of formalised mathematics
- Started in 2017; now contains over 200 000 theorems

3 Writing our first proofs

4 Review of syntax

- `example`
Used to introduce a new proposition to prove.
- `rfl`
Prove a proposition by definition.
- `exact h`
Prove a proposition by applying hypothesis `h`.
- `rw [h]`
Rewrite the goal using hypothesis `h`.
- Comments can be written with: `-- here is a comment.`
- `#check`
Check the type of an expression.
`#check 1` gives `1:ℕ`.
`#check (1 + 1 = 2)` gives `(1 + 1 = 2):Prop`.
- `constructor`
Transform the goal `P and Q` into two goals, `P` and `Q`.
- `sorry`
Temporarily prove any goal (and raise a warning).
- `have`
Introduce a new sub-goal to prove.

5 Basics of type-theoretic logic

- The Curry–Howard correspondence gives an isomorphism between models for *type theory* and *logic*.

Type theory	Logic
Type T	Proposition P
Element of type $x:T$	Proof of proposition $h:P$
$n:\mathbb{N}$	$\text{rfl}:(1 + 1 = 2)$

- The proposition **True** has a proof `trivial:True`.
- The proposition **False** has no proofs!
- If P and Q are propositions, then a proof of $P \implies Q$ is a function mapping proofs of P to proofs of Q .
- A proof of P **and** Q is a pair of proofs, one for P and one for Q : $(hP, hQ):(P \text{ and } Q)$ where $hP:P$ and $hQ:Q$.
- A proof of **not** P is a function mapping proofs of P to proofs of **False**.
- P **or** Q is rather more complicated: we may prove P **or** Q without having a proof for either P or Q . For example:

$$\exists p, q \in \mathbb{R} \setminus \mathbb{Q} : p^q \in \mathbb{Q}.$$

Proof. Consider $x := \sqrt{2}^{\sqrt{2}}$ and $x^{\sqrt{2}}$.

6 Infinitude of the primes

We present a famous result and proof due to Euclid, published in the Elements, around 300 BCE.

Theorem (Infinitude of the primes). *For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.*

Proof. Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m .

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.
- Suppose for contradiction that we do not have $p > n$.
- Then $p \leq n$.
- Note that $p > 0$ because p is a factor of m .
- We have $p \mid m$ by definition of p .
- Also, $p \mid n!$ because $p \leq n$ by assumption and $p > 0$.
- So $p \mid 1$ by modular arithmetic.
- Thus, $p = 1$.
- But 1 is not prime ✖. □

7 Getting help

- `#check`
Check the type of an expression. Remember: propositions and proofs have types and can be checked!
For example, `#check Nat.eq_one_of_dvd_one`
- `#eval`
Evaluate some Lean code.
For example, `#eval Nat.minFac 1`
- Advanced tactics for finding proofs:
`aesop`, `omega`, `linarith`, `ring`, `grind`
- <https://leansearch.net/>
Search Mathlib using natural language.
- <https://github.com/leanprover-community/mathlib4>
The Mathlib code repository.
- <https://lean-lang.org/>
Core documentation for Lean.

8 A simple limit theorem